

LAPORAN PRAKTIKUM

Praktikum 4 - INHERITANCE, GENERALIZATION & OVERRIDING

Mata Kuliah Pemrograman Berorientasi Objek · D3 Teknik Informatika

Identitas	Isian
Nama	Rizky Yanuar Irawan
NIM	251511029
Kelas	2A D3
Tanggal Mulai	30 September 2026
Tanggal Pengumpulan	07 Oktober 2026
URL Repository	https://github.com/RizkyYanuar/pbo-praktikum4.git

1. Praktikum I - Circle dan Cylinder

1. Step 1 - Bangun Circle versi paling sederhana

a. Checkpoint 1

State circle adalah radius dan color.

Behavior circle adalah toString().

Perbedaan state dari dua objek yang dibuat: Pada objek pertama, radius secara default adalah 1.0 tetapi pada objek kedua, radius bersifat dinamis mengikuti inputan user dengan tipe data double.

2. Step 2 - Lengkapi encapsulation Circle

a. Checkpoint 2

Code diluar circle tidak mengubah radius secara langsung karena variabel radius dibuat private sehingga hanya bisa diakses oleh objek itu sendiri (melalui setter dan getter).

Peran method public terhadap state private adalah method public menjadi gerbang bagi state private untuk diubah nilainya oleh objek luar

3. Step 3 - Prediksi sebelum membuat Cylinder

a. Apakah radius dan color perlu ditulis ulang sebagai field Cylinder?

Tidak perlu, Ketika pemanggilan constructor circle sudah otomatis dibuat.

b. Behavior apa dari Circle yang masih berguna untuk Cylinder?

getArea yang digunakan untuk menghitung luas alas lingkaran

toString() untuk mencetak objek

c. State apa yang hanya dimiliki Cylinder?

Height (tinggi cylinder)

4. Step 4 - Buat Cylinder sebagai subclass Circle

Cylinder sudah ditambahkan.

5. Step 5 - Uji inherited method

a. Checkpoint 3: Jelaskan berdasarkan code: mengapa c3.getRadius() dan c3.getArea() dapat dipanggil padahal method tersebut tidak ditulis ulang pada Cylinder?

getRadius() dan getArea() dapat dipanggil karena kedua method tersebut diwarisi dari parent-nya, yaitu Circle.

6. Step 6 - Amati constructor chain dan fungsi super(...)

a. Hasil urutan output constructor

```
PS D:\POLBAN\SEMESTER 3\PB0\pertemuan4> & 'C:\Program Files\Java\jdk-26.0.2.1\bin\
' 'C:\Users\Rizky Yanuar Irawan\AppData\Roaming\Code\User\workspaceStorage\69d99739
ylinder'
Circle(double) dipanggil
Cylinder(double,double) dipanggil
PS D:\POLBAN\SEMESTER 3\PB0\pertemuan4>
```

Hasilnya adalah constructor Circle(double) dipanggil lebih dulu lalu Cylinder(double, double) setelahnya.

- b. Hasil hapus super(radius)

```
PS D:\POLBAN\SEMESTER 3\PBO\pertemuan4> d:; cd 'd:\POLBAN\SEMESTER 3\PBO\pertemuan4'
'-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\user\Downloads\at.java\jdt_ws\pertemuan4_85f990b3\bin' 'TestCylinder'
Cylinder(double,double) dipanggil
PS D:\POLBAN\SEMESTER 3\PBO\pertemuan4>
```

Hasilnya adalah constructor Circle(double) tidak terpanggil. Sehingga apabila di print hasil pembuatan cylindernya maka radius cylinder tersebut akan menjadi default, yaitu 1.0

```
Cylinder(double,double) dipanggil
c3
radius = 1.0
height = 10.0
base area = 3.141592653589793
volume = 31.41592653589793
PS D:\POLBAN\SEMESTER 3\PBO\pertemuan4>
```

(radius default)

```
PS D:\POLBAN\SEMESTER 3\PBO\pertemuan4> d:; cd 'd:\POLBAN\SEMESTER 3\PBO\pertemuan4'
'-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\user\Downloads\at.java\jdt_ws\pertemuan4_85f990b3\bin' 'TestCylinder'
Circle(double) dipanggil
Cylinder(double,double) dipanggil
c3
radius = 2.0
height = 10.0
base area = 12.566370614359172
volume = 125.66370614359172
PS D:\POLBAN\SEMESTER 3\PBO\pertemuan4>
```

(radius buatan)

- c. Checkpoint 4: Apa yang dikerjakan lebih dahulu saat object Cylinder dibuat: bagian Circle atau bagian Cylinder? Bukti apa dari output Anda?

Yang dikerjakan lebih dahulu adalah bagian Circle karena dapat dilihat bahwa Ketika melakukan print, yang muncul pertama kali adalah output circle(double) dipanggil.

7. Step 7 - Uji batas encapsulation pada subclass

- a. Pesan error compiler

```

PS D:\POLBAN\SEMESTER 3\PBO\pertemuan4> d:; cd 'd:\POLBAN\SEMESTER 3\PBO\pertemuan4'; & 'C:\Program Files\Java\jdk-9.0.4\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Rizky Yanuar Irawan\AppData\Roaming\Java\jdk-9.0.4\bin\java\jdt_ws\pertemuan4_85f990b3\bin' 'TestCylinder'
Exception in thread "main" java.lang.Error: Unresolved compilation problem:
    The field Circle.radius is not visible
    at Cylinder.cobaAksesRadius(Cylinder.java:32)
    at TestCylinder.main(TestCylinder.java:9)
PS D:\POLBAN\SEMESTER 3\PBO\pertemuan4>

```

- b. Checkpoint 5: Lengkapi kalimat: extends membuat subclass mewarisi behavior yang dapat diakses, tetapi tidak membuat field private superclass menjadi dapat diakses langsung melalui subclassnya

8. Step 8 - Override getArea(), lalu temukan bug pada getVolume()

- a. Perbandingan getArea()

Sebelum

```

c3
radius = 2.0
height = 10.0
base area = 12.566370614359172
volume = 125.66370614359172

```

Sesudah

```

c3
radius = 2.0
height = 10.0
base area = 150.79644737231007
volume = 1507.9644737231006

```

Dapat dilihat bahwa hasil dari keduanya menjadi berbeda meskipun radius dan heightnya sama.

- b. Hasil pemeriksaan getVolume()

Setelah meng-override getArea(), nilai base area yang sebelumnya (12.5) adalah nilai dari luas alas lingkarannya, sedangkan setelah di override, nilai base area setelahnya (150) adalah nilai dari luas permukaan tabung. Sehingga Ketika luas permukaan dikali tinggi tabung maka nilai volume tidak valid. Nilai volume yang valid adalah nilai volume pada getVolume() sebelum di override oleh getArea() di Cylinder.

- c. getArea() yang dipanggil dari getVolume()

getArea() yang dipanggil dari getVolume() adalah getArea yang berasal dari Cylinder (overriding).

- d. Hasil perbaikan getVolume()

```

-XX:+ShowCodeDetailsInExceptionMessages -cp C:\Users\Rizky Yanuar Irawan\AppData\Roaming\Java\jdk-9.0.4\bin\java\jdt_ws\pertemuan4_85f990b3\bin' 'TestCylinder'
c3
radius = 2.0
height = 10.0
base area = 150.79644737231007
volume = 125.66370614359172

```

Hasilnya sesuai dimana nilai volume berasal dari getArea() superclassnya.

- e. Checkpoint 6: getArea() pada object Cylinder sekarang berbeda dari getArea() pada Circle. super.getArea() dipakai Ketika code di subclass secara sengaja ingin menggunakan implementasi milik superclass.

9. Step 9 - Override toString() dan uji @Override

- a. Cetak

```

PS D:\POLBAN\SEMESTER 3\PBO\pertemuan4> d:; cd 'd:\POLBAN\SEMESTER 3\PBO\pertemuan4'
'-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Rizky Yanuar I\at.java\jdt_ws\pertemuan4_85f990b3\bin' 'TestCylinder'
Cylinder: subclass of Circle[radius=2.0, color=red], height=10.0
PS D:\POLBAN\SEMESTER 3\PBO\pertemuan4>

```

Hasilnya mengikuti apa yang di override kan di cylinder.

b. Error

```

PS D:\POLBAN\SEMESTER 3\PBO\pertemuan4> d:; cd 'd:\POLBAN\SEMESTER 3\PBO\pertemuan4'
'-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Rizky Yanuar I\at.java\jdt_ws\pertemuan4_85f990b3\bin' 'TestCylinder'
Circle[radius=2.0, color=red]
PS D:\POLBAN\SEMESTER 3\PBO\pertemuan4>

```

Sebenarnya di output tidak error, karena pemanggilan fungsi toString terjadi di level superclass.

Tetapi Ketika di lihat pada file cylinder, ada pesan error

```

r.getArea() * height;
The method toString() of type Cylinder must override or implement a supertype
method Java(67109498)
The method toString() of type Cylinder must override or implement a supertype
method Java(67109498)
String cylinder.toString()
Source: pertemuan4_85f990b3

```

Yang artinya method override harus implementasi dari superclassnya.

c. Perbedaan Ketika @override dihapus

```

PS D:\POLBAN\SEMESTER 3\PBO\pertemuan4> d:; cd 'd:\POLBAN\SEMESTER 3\PBO\pertemuan4'
'-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Rizky Yanuar I\at.java\jdt_ws\pertemuan4_85f990b3\bin' 'TestCylinder'
Cylinder: subclass of Circle[radius=2.0, color=red], height=10.0
PS D:\POLBAN\SEMESTER 3\PBO\pertemuan4>

```

Hasil output ternyata tetap sama

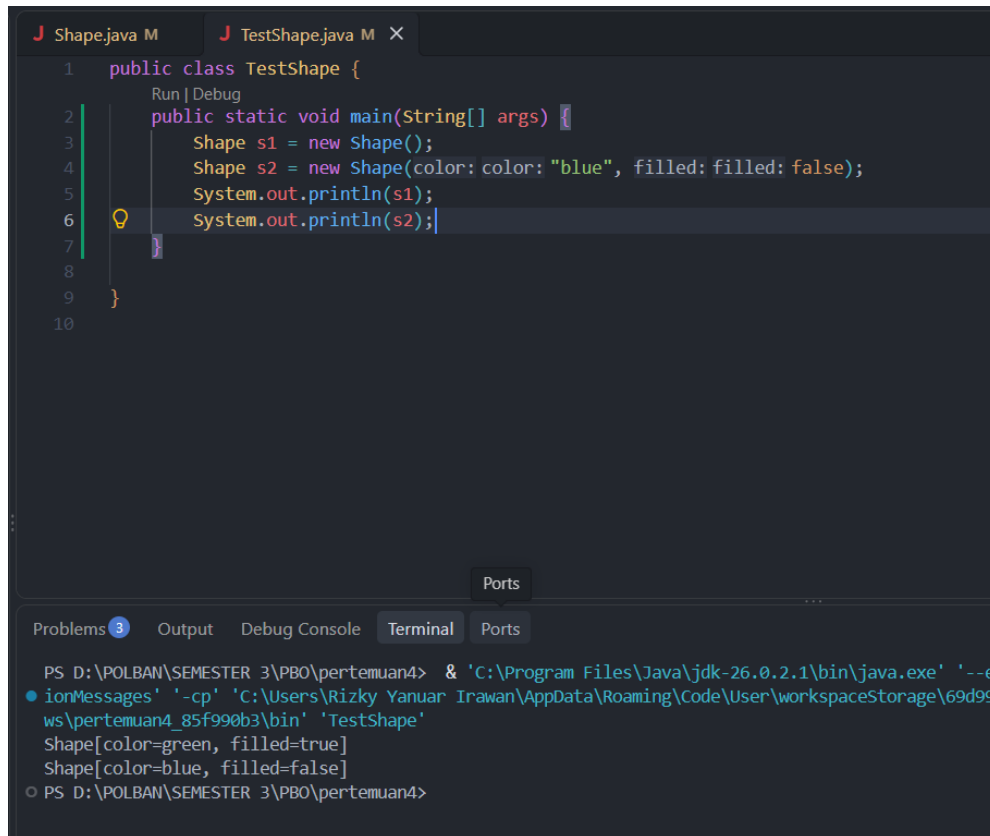
d. Checkpoint 7: Tuliskan dengan bahasa Anda: @Override tidak membuat inheritance terjadi. Lalu apa kegunaannya?

Sebagai penanda bahwa fungsi yang di definisikan di bawahnya adalah fungsi subclass yang akan menimpa fungsi superclassnya. Bisa digunakan juga untuk validasi kebenaran, dimana jika kita bermaksud untuk melakukan overriding tetapi nama fungsinya salah atau tidak sesuai dengan yang ada di superclass maka compiler akan menunjukkan pesan error.

2. Praktikum II - Shape, Circle, Rectangle, dan Square

10. Step 10 - Buat superclass Shape

- a. Hasil instansiasi objek shape



```
Shape.java M TestShape.java M X
1 public class TestShape {
2     Run | Debug
3     public static void main(String[] args) {
4         Shape s1 = new Shape();
5         Shape s2 = new Shape(color: "blue", filled: filled: false);
6         System.out.println(s1);
7         System.out.println(s2);
8     }
9 }
10

Problems 3 Output Debug Console Terminal Ports
PS D:\POLBAN\SEMESTER 3\PB0\pertemuan4> & 'C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe' '--er
ionMessages' '-cp' 'C:\Users\Rizky Yanuar Irawan\AppData\Roaming\Code\User\workspaceStorage\69d997
ws\pertemuan4_85f990b3\bin' 'TestShape'
Shape[color=green, filled=true]
Shape[color=blue, filled=false]
PS D:\POLBAN\SEMESTER 3\PB0\pertemuan4>
```

- b. Checkpoint 8: State apa yang cukup umum sehingga masuk akal dimiliki Circle maupun Rectangle?

Yang cukup umum adalah yang menjadi state dari shape, yaitu color dan filled. Color digunakan untuk pewarnaan objek, sedangkan filled digunakan untuk parameter apakah shape tersebut diisi warna penuh atau tidak.

11. Step 11 - Buat Circle sebagai specialization dari Shape

- a. Hasil Cetak area, perimeter, color, filled, dan toString().

```

Run | Debug
2   public static void main(String[] args) {
3       // Shape s1 = new Shape();
4       // Shape s2 = new Shape("blue", false);
5       // System.out.println(s1);
6       // System.out.println(s2);
7       Circle c1 = new Circle();
8       Circle c2 = new Circle(radius: radius: 2.0);
9       Circle c3 = new Circle(radius: radius: 3.0, color: color: "yellow", filled: filled: true);
10      System.out.println(c1);
11      System.out.println(c2);
12      System.out.println(c3);
13  }
14
15  }
16

```

Problems 4 Output Debug Console Terminal Ports

```

PS D:\POLBAN\SEMESTER 3\PBO\pertemuan4> & 'C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe' '-enable-preview' '-cp' 'C:\Users\Rizky Yanuar Irawan\AppData\Roaming\Code\User\workspaceStorage\69d9973993e82d0181a86072def6601b\redhat.java\jdt_ws\pertemuan4_85f990b3\bin' 'TestShape'
Shape[color=green, filled=true]
Shape[color=blue, filled=false]
PS D:\POLBAN\SEMESTER 3\PBO\pertemuan4> d; cd 'd:\POLBAN\SEMESTER 3\PBO\pertemuan4'; & 'C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe' '-enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Rizky Yanuar Irawan\AppData\Roaming\Code\User\workspaceStorage\69d9973993e82d0181a86072def6601b\redhat.java\jdt_ws\pertemuan4_85f990b3\bin' 'TestShape'
Circle[radius=1.0, Shape[color=green, filled=true]]
Circle[radius=2.0, Shape[color=green, filled=true]]
Circle[radius=3.0, Shape[color=yellow, filled=true]]
PS D:\POLBAN\SEMESTER 3\PBO\pertemuan4>

```

- b. Checkpoint 9: Pisahkan daftar berikut menjadi dua: behavior yang berasal dari Shape dan behavior yang spesifik milik Circle.

Behavior milik shape: getColor(), setColor(), isFilled(), setFilled(), dan toString()

Behavior milik circle: getRadius(), setRadius(), getArea(), getPerimeter()

12. Step 12 - Buat Rectangle sebagai specialization dari Shape

- a. Hasil instantiate dan cetak

```

12      // System.out.println(c3);
13      Rectangle r1 = new Rectangle();
14      Rectangle r2 = new Rectangle(width: width: 2.0, length: length: 3.0);
15      Rectangle r3 = new Rectangle(width: width: 4.0, length: length: 5.0, color: color: "black", filled: false);
16      System.out.println(r1);
17      System.out.println(r2);
18      System.out.println(r3);
19
20

```

Problems 5 Output Debug Console Terminal Ports

```

PS D:\POLBAN\SEMESTER 3\PBO\pertemuan4> d; cd 'd:\POLBAN\SEMESTER 3\PBO\pertemuan4'; & 'C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe' '-enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Rizky Yanuar Irawan\AppData\Roaming\Code\User\workspaceStorage\69d9973993e82d0181a86072def6601b\redhat.java\jdt_ws\pertemuan4_85f990b3\bin' 'TestShape'
Rectangle[width=1.0, length=1.0, Shape[color=green, filled=true]]
Rectangle[width=2.0, length=3.0, Shape[color=green, filled=true]]
Rectangle[width=4.0, length=5.0, Shape[color=black, filled=false]]
PS D:\POLBAN\SEMESTER 3\PBO\pertemuan4>

```

- b. Checkpoint 10: Apa kemiripan Circle dan Rectangle yang sudah ditarik ke Shape? Apa yang sengaja tetap berbeda?

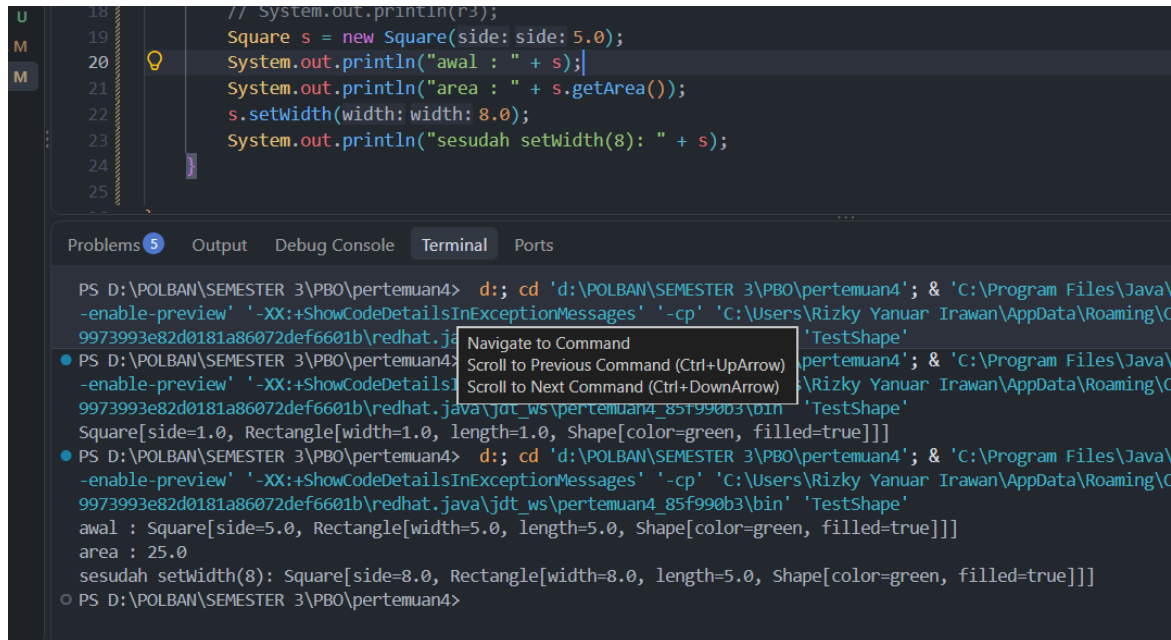
Kemiripan dari circle dan rectangle adalah keduanya merupakan subclass dari superclass yang sama, yaitu superclass shape. Yang tetap dibedakan adalah constructornya, dimana rectangle memiliki width dan length, sedangkan circle hanya memiliki radius. Dan behavior dari keduanya juga berbeda.

13. Step 13 - Buat Square dari Rectangle

```
Rectangle[width=4.0, length=5.0, Shape[color=black, filled=false]]
● PS D:\POLBAN\SEMESTER 3\PBO\pertemuan4> d:;; cd 'd:\POLBAN\SEMESTER 3\PBO\pertemuan4'; & 'C:\Program Files\Java\jdk-9.0.4\bin\java.exe' '-enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Rizky Yanuar Irawan\AppData\Roaming\OpenJDK\jre-9.0.4\bin\TestShape' Square[side=1.0, Rectangle[width=1.0, length=1.0, Shape[color=green, filled=true]]]
○ PS D:\POLBAN\SEMESTER 3\PBO\pertemuan4>
```

14. Step 14 - Temukan pelanggaran aturan Square

- a. Hasil width dan length setelah setWidth(8).



```
// System.out.println(r3);
19 Square s = new Square(side: side: 5.0);
20 System.out.println("awal : " + s);
21 System.out.println("area : " + s.getArea());
22 s.setWidth(width: width: 8.0);
23 System.out.println("sesudah setWidth(8): " + s);
24
25
```

Problems 5 Output Debug Console Terminal Ports

```
PS D:\POLBAN\SEMESTER 3\PBO\pertemuan4> d:;; cd 'd:\POLBAN\SEMESTER 3\PBO\pertemuan4'; & 'C:\Program Files\Java\jdk-9.0.4\bin\java.exe' '-enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Rizky Yanuar Irawan\AppData\Roaming\OpenJDK\jre-9.0.4\bin\TestShape' Square[side=1.0, Rectangle[width=1.0, length=1.0, Shape[color=green, filled=true]]]
● PS D:\POLBAN\SEMESTER 3\PBO\pertemuan4> d:;; cd 'd:\POLBAN\SEMESTER 3\PBO\pertemuan4'; & 'C:\Program Files\Java\jdk-9.0.4\bin\java.exe' '-enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Rizky Yanuar Irawan\AppData\Roaming\OpenJDK\jre-9.0.4\bin\TestShape' Square[side=1.0, Rectangle[width=1.0, length=1.0, Shape[color=green, filled=true]]]
● PS D:\POLBAN\SEMESTER 3\PBO\pertemuan4> d:;; cd 'd:\POLBAN\SEMESTER 3\PBO\pertemuan4'; & 'C:\Program Files\Java\jdk-9.0.4\bin\java.exe' '-enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Rizky Yanuar Irawan\AppData\Roaming\OpenJDK\jre-9.0.4\bin\TestShape' Square[side=1.0, Rectangle[width=1.0, length=1.0, Shape[color=green, filled=true]]]
awal : Square[side=5.0, Rectangle[width=5.0, length=5.0, Shape[color=green, filled=true]]]
area : 25.0
sesudah setWidth(8): Square[side=8.0, Rectangle[width=8.0, length=5.0, Shape[color=green, filled=true]]]
○ PS D:\POLBAN\SEMESTER 3\PBO\pertemuan4>
```

- b. apakah object tersebut masih merepresentasikan Square?

Tidak, karena lengthnya masih 5.0, sehingga tidak bisa disebut persegi. Fungsi setWidth yang terdapat di rectangle hanya mengubah nilai widthnya tapi tidak dengan length. Mungkin solusinya bisa dilakukan overriding di subclass square untuk menambahkan perubahan lengthnya agar sama dengan width yang diinputkan.

- c. Checkpoint 11: Inheritance dapat memberikan behavior yang secara teknis tersedia tetapi tidak selalu sesuai dengan aturan subclass. Di sinilah overriding dapat menjaga aturan object yang lebih spesifik.

15. Step 15 - Override setter agar Square tetap valid

- a. Hasil test step 14

```
9973993e82d0181a86072def6601b\redhat.java\jdt_ws\pertemuan4_85f990b3\bin' 'TestShape'
awal : Square[side=5.0, Rectangle[width=5.0, length=5.0, Shape[color=green, filled=true]]]
area : 25.0
sesudah setWidth(8): Square[side=8.0, Rectangle[width=8.0, length=8.0, Shape[color=green, filled=true]]]
PS D:\POLBAN\SEMESTER 3\PBO\pertemuan4>
```

Hasil menunjukkan keduanya (width dan length) memiliki nilai yang sama sehingga dapat disebut sebagai persegi. Sebenarnya, overriding kedua fungsi itu tidak terlalu diharuskan karena kan sudah ada fungsi setSide() di subclass Square.

- b. Hasil test s.setLength(3.0), lalu cek width dan length.

```

9973993e82d0181a80072d0f6001b (rednat.java) [jdt_ws\pertemuan4_85199003\bin - testShape
awal : Square[side=5.0, Rectangle[width=5.0, length=5.0, Shape[color=green, filled=true]]]
area : 25.0
sesudah setWidth(8): Square[side=8.0, Rectangle[width=8.0, length=8.0, Shape[color=green, filled=true]]]
sesudah setLength(3): Square[side=3.0, Rectangle[width=3.0, length=3.0, Shape[color=green, filled=true]]]
PS D:\POLBAN\SEMESTER 3\PBO\pertemuan4>

```

SetLength juga menghasilkan width dan length yang sama, karena fungsi ini sudah di override di subclass square.

- c. Uji getArea() dan getPerimeter(). Apakah perlu dioverride di Square? Jangan menebak; buktikan dari output.

```

9973993e82d0181a80072d0f6001b (rednat.java) [jdt_ws\pertemuan4_85199003\bin - testShape
awal : Square[side=5.0, Rectangle[width=5.0, length=5.0, Shape[color=green, filled=true]]]
area : 25.0
sesudah setWidth(8): Square[side=8.0, Rectangle[width=8.0, length=8.0, Shape[color=green, filled=true]]]
sesudah setLength(3): Square[side=3.0, Rectangle[width=3.0, length=3.0, Shape[color=green, filled=true]]]
area : 9.0
perimeter : 12.0
PS D:\POLBAN\SEMESTER 3\PBO\pertemuan4>

```

Hasil menunjukkan nilai yang benar sehingga getArea dan getPerimeter tidak perlu di override.

- d. Checkpoint 12: Jelaskan mengapa setWidth()/setLength() perlu dioverride, tetapi getArea()/getPerimeter() dapat tetap memakai implementasi Rectangle.

Kenapa tidak perlu di override karena karena yang penting nilai width dan lengthnya sama dulu agar bisa disebut persegi. Rumus bangun datar segi empat kan sama.

16. Step 16 - Test hierarchy secara utuh

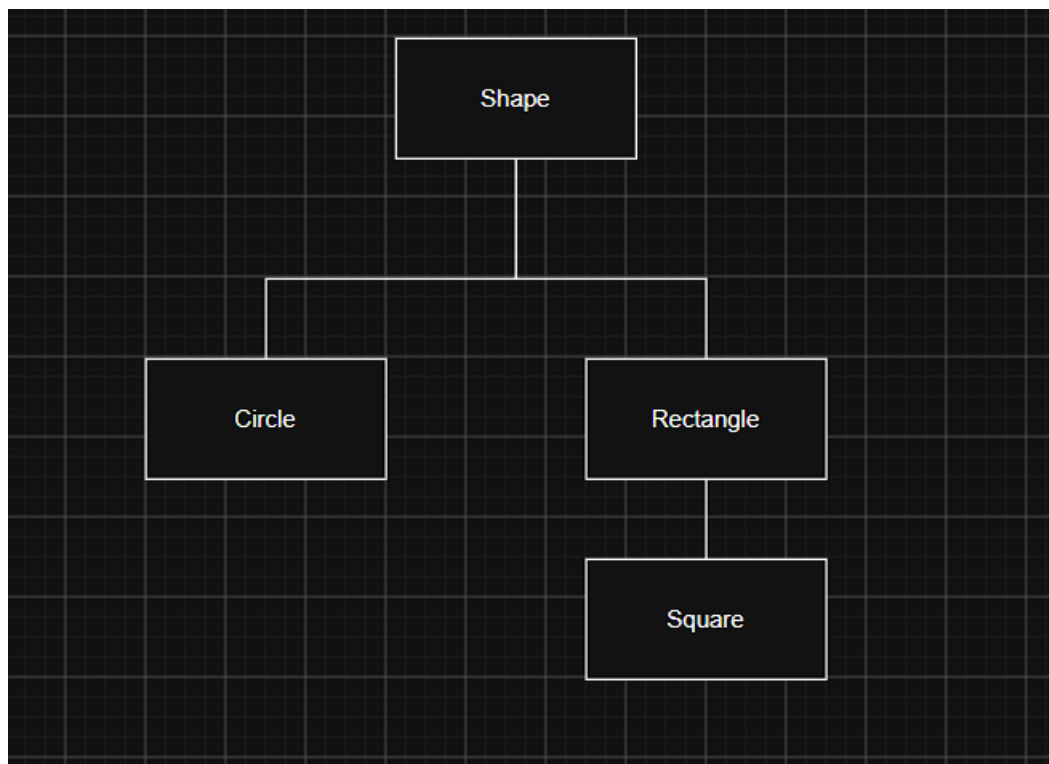
- a. Hasil

```

9973993e82d0181a80072d0f6001b (rednat.java) [jdt_ws\pertemuan4_85199003\bin - testShape
Shape[color=black, filled=false]
Circle[radius=2.0, Shape[color=blue, filled=true]] area=12.566370614359172
Rectangle[width=4.0, length=4.0, Shape[color=yellow, filled=true]] area=12.0
Square[side=5.0, Rectangle[width=5.0, length=5.0, Shape[color=green, filled=true]]] area=25.0
PS D:\POLBAN\SEMESTER 3\PBO\pertemuan4>

```

- b. Diagram Hirarki



3. Kembangkan Kasus Tugas Sebelumnya

17. Step 17 - Cari kandidat generalization

- a. Daftar minimal dua class yang memiliki state/behavior mirip.

Class yang memiliki state mirip adalah jenis dari MenuItem, dimana di praktikum sebelumnya MenuItem itu memuat makanan dan minuman, keduanya adalah merupakan item yang bisa dipesan, namun jenisnya berbeda.

- b. Tuliskan state/behavior yang sama.

- State yang sama adalah kode, Nama, harga, tersedia
- Behavior yang sama adalah getCode(), getName(), getHarga(), isTersedia(), setTersedia()

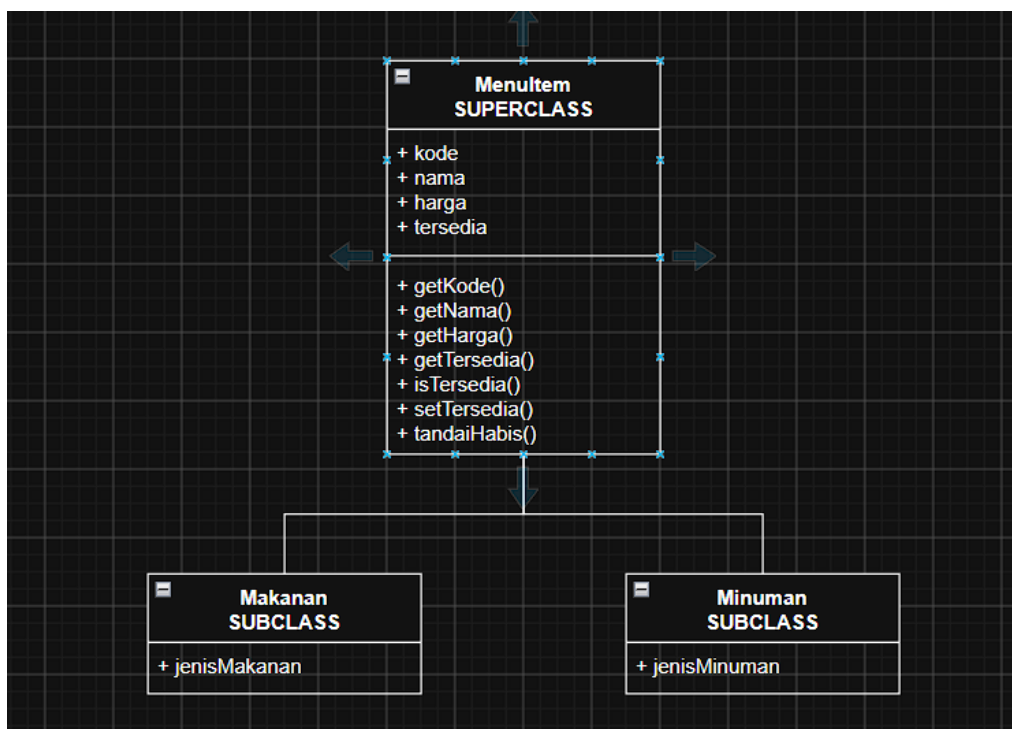
- c. Tuliskan perbedaan masing-masing class.

Perbedaannya ada di jenisnya, yaitu makanan atau minuman.

- d. Buat kalimat: "Subclass adalah sebuah Superclass". Jika kalimat terasa tidak masuk akal, jangan gunakan inheritance.

- Makanan adalah MenuItem dan Minuman adalah MenuItem, sehingga inheritance bisa digunakan.

18. Step 18 - Gambar sebelum coding



19. Step 19 - Implementasikan perubahan

- Minimal 1 superclass dan 2 subclass yang masuk akal.

Superclass:

```
package kasus_sebelumnya.Model;

public class MenuItem {
    private String kode;
    private String nama;
    private int harga;
    private boolean tersedia;

    public MenuItem(String kode, String nama, int harga) {
        this.kode = kode;
        this.nama = nama;
        this.harga = harga;
        this.tersedia = true;
    }

    public String getKode() {
        return kode;
    }

    public String getNama() {
        return nama;
    }

    public int getHarga() {
        return harga;
    }

    public boolean isTersedia() {
        return tersedia;
    }

    public void setTersedia(boolean tersedia) {
        this.tersedia = tersedia;
    }

    @Override
    public String toString() {
        return "MenuItem[kode=" + kode +
            ", nama=" + nama +
            ", harga=" + harga +
            ", tersedia=" + tersedia + "];"
    }
}
```

Subclass:

```
package kasus_sebelumnya.Model;

public class Minuman extends MenuItem {
    private String jenisMinuman;

    public Minuman(String kode, String nama, int harga, String jenisMinuman) {
        super(kode, nama, harga);
        this.jenisMinuman = jenisMinuman;
    }

    public String getJenisMinuman() {
        return jenisMinuman;
    }

    @Override
    public String toString() {
        return "Minuman[" +
            super.toString() +
            ", jenis=" + jenisMinuman + "];"
    }
}
```

```

package kasus_sebelumnya.Model;

public class Makanan extends MenuItem {
    private String jenisMakanan;

    public Makanan(String kode, String nama, int harga, String jenisMakanan) {
        super(kode, nama, harga);
        this.jenisMakanan = jenisMakanan;
    }

    public String getJenisMakanan() {
        return jenisMakanan;
    }

    @Override
    public String toString() {
        return "Makanan[" +
            super.toString() +
            ", jenis=" + jenisMakanan + "]";
    }
}

```

- b. Subclass menggunakan extends.

Class makanan dan minuman sudah menggunakan extends.

- c. Constructor subclass memanggil super(...).

Sudah dipanggil, dan digunakan untuk set kode, nama, harga

- d. Minimal 1 method dioverride menggunakan @Override.

Override toString()

- e. Minimal 1 override memanfaatkan super.method(...) secara bermakna, atau tulis alasan teknis jika memang tidak dibutuhkan.

Override toString() digunakan untuk mendapatkan informasi dari superclass

- f. Encapsulation tetap dijaga; field private tidak diubah menjadi public.

Atribut kode, nama, harga, tersedia masih bersifat private

- g. Program utama membuat object dari minimal dua subclass dan menunjukkan output/behavior yang berbeda.

```

PS D:\POLBAN\SEMESTER 3\PBO\pertemuan4> d:; cd 'd:\POLBAN\SEMESTER 3\PBO\pertemuan4'; & 'C:\Program Files\Java\jdk-11.0.2\bin\java.exe' -cp 'C:\Users\Rizky Yanuar Irawan\AppData\Roaming\Code\User\workspaceStorage\69f990b3\bin' 'kasus_sebelumnya.App.Mahasiswa'
Kopi Susu tersedia: true
Pemesan Pertama: Rizky Yanuar Irawan
Pemesan Kedua: Kareem Nul Mustofa
Nomor pesanan pertama: 1
Nomor pesanan kedua: 2
Nomor pesanan ketiga: 3
Jumlah pesanan dibuat: 3
Makanan[MenuItem[kode=M01, nama=Nasi Goreng, harga=18000, tersedia=true], jenis=Makanan Utama]
Minuman[MenuItem[kode=M02, nama=Kopi Susu, harga=12000, tersedia=true], jenis=Kopi]
Minuman[MenuItem[kode=M03, nama=Matcha Latte, harga=25000, tersedia=true], jenis=Latte Series]
PS D:\POLBAN\SEMESTER 3\PBO\pertemuan4>

```

- h. Fungsi utama tugas sebelumnya tetap berjalan setelah refactor.

Berjalan dengan lancar.

20. Step 20 - Uji sebelum menyatakan selesai

ID	SKENARIO	EXPECTED RESULT	STATUS	BUKTI
T4-01	Object subclass dibuat melalui constructor	Bagian superclass dan subclass terinisialisasi benar	PASS	<pre> Makanan nasi = new Makanan(kode: kode: "M01", nama: nama: "Nasi Goreng", harga: harga: 18000, jenisMakanan: jenisMakanan: "Makanan Utama"); Minuman kopi = new Minuman(kode: kode: "M02", nama: nama: "Kopi Susu", harga: harga: 12000, jenisMinuman: jenisMinuman: "Kopi"); Minuman matcha = new Minuman(kode: kode: "M03", nama: nama: "Matcha Latte", harga: harga: 25000, jenisMinuman: jenisMinuman: "Latte Series"); </pre>
T4-02	Method warisan dipanggil dari subclass	Behavior superclass tersedia tanpa copy code	PASS	<pre> System.out.println(nasi.getNama() + " tersedia: " + nasi.isTersedia()); System.out.println(kopi.getNama() + " tersedia: " + kopi.isTersedia()); System.out.println(kopi.getNama() + " tersedia: " + kopi.isTersedia()); </pre>
T4-03	Method override dipanggil	Behavior subclass yang dijalankan	PASS	<pre> @Override public String toString() { return "Minuman[" + super.toString() + ", jenis=" + jenisMinuman + "];" } </pre>
T4-04	Override memakai super.method(...)	Bagian behavior superclass tetap digunakan	PASS	<pre> @Override public String toString() { return "Minuman[" + super.toString() + ", jenis=" + jenisMinuman + "];" } </pre>
T4-05	Akses state mengikuti encapsulation	Tidak mengakses field private secara langsung	PASS	<pre> public class MenuItem { private String kode; private String nama; private int harga; private boolean tersedia; } </pre>
T4-07	Dua subclass diuji	Perbedaan behavior terlihat pada output	PASS	<pre> Makanan[MenuItem[kode=M01, nama=Nasi Goreng, harga=18000, tersedia=true], jenis=Makanan Utama] Minuman[MenuItem[kode=M02, nama=Kopi Susu, harga=12000, tersedia=true], jenis=Kopi] Minuman[MenuItem[kode=M03, nama=Matcha Latte, harga=25000, tersedia=true], jenis=Latte Series] </pre>
T4-07	Regression tugas lama	Fungsi utama sebelumnya tetap berjalan	PASS	Di point g pada nomor 19.

21. Self Assesment

Pernyataan	Skor 1-5
Saya dapat menjelaskan extends dari code yang saya jalankan.	5

Saya dapat membedakan inherited method dan overridden method.	5
Saya memahami mengapa constructor subclass memanggil super(...).	5
Saya dapat menggunakan super.method(...) dengan alasan yang tepat.	5
Saya dapat menjaga encapsulation pada hierarchy class.	5
Saya dapat menentukan kapan inheritance masuk akal pada kasus sendiri.	5

22. Refleksi

1. Apa yang membuat Circle layak menjadi class yang lebih umum daripada Cylinder?

Karena cylinder memiliki karakteristik dasar dari sebuah circle, yaitu radius

2. Berdasarkan Step 5, apa arti praktis extends bagi object Cylinder?

Sebuah subclass dapat menggunakan fungsi yang di definisikan di superclassnya

3. Mengapa constructor Circle dijalankan ketika Anda membuat object Cylinder?

Karena untuk membuat cylinder, bagian superclassnya (circle) perlu diisi terlebih dahulu

4. Mengapa getVolume() menjadi salah setelah getArea() dioverride?

Karena setelah getArea() dioverride di cylinder, fungsi getArea() yang dipanggil mengembalikan nilai luas permukaan tabung, bukan luas alas lingkarannya

5. Apa yang secara eksplisit Anda minta ketika menulis super.getArea() atau super.toString()?

Tolong eksekusi fungsi getArea() atau toString() dari superclass saya

6. Mengapa Square perlu mengubah setter yang diwarisi dari Rectangle?

Karena rectangle memiliki width dan length, sedangkan square ukuran di semua sisinya sama.

7. Apakah setiap dua class yang mirip harus menggunakan inheritance? Jelaskan dari salah satu kasus Anda.

Tidak selalu karena Inheritance digunakan jika terdapat hubungan "is-a". Contoh, square dan rectangle dapat menggunakan inheritance karena square adalah sebuah rectangle.

8. Lengkapi dengan bahasa Anda sendiri: "Inheritance bukan sekadar mengurangi duplikasi, tetapi..."

Digunakan untuk menyatakan hubungan antar class